

CMPT 310 - Project Milestone 2

Project Recap

Our project is an AI agent that learns to play the game 2048 autonomously. The input is the current 4×4 game board, and the output is one of four possible actions (up, down, left, or right). We use reinforcement learning as our primary AI approach, integrated with an Expectimax search agent. A Random agent and the original heuristic-based Expectimax agent from Milestone 1 are used as baseline methods for comparison.

Accomplishments with Proof

Accomplishment 1: Implemented N-Tuple Network Value Function

To enable reinforcement learning, we replaced the manually designed heuristic evaluation function from our Expectimax agent with a learned value function. We implemented an N-tuple network that estimates the value of a board by summing lookup table entries selected by groups of cells, so the value $V(s)$ of a position is learned from experience rather than designed by hand, with higher values meaning a position expected to earn more future reward.

Proof: code excerpts from `agents/n_tuple_network.py`

```
15 Coordinate = Tuple[int, int]
14 TuplePattern = Tuple[Coordinate, ...]
13 PathLike = Union[str, "Path"]
12
11
10 def _default_tuples() -> Tuple[TuplePattern, ...]:
9     rows = [tuple((row, col) for col in range(BOARD_SIZE)) for row in range(BOARD_SIZE)]
8     columns = [tuple((row, col) for row in range(BOARD_SIZE)) for col in range(BOARD_SIZE)]
7     squares = [
6         tuple((row + dr, col + dc) for dr in range(2) for dc in range(2))
5         for row in range(BOARD_SIZE - 1)
4         for col in range(BOARD_SIZE - 1)
3     ]
2     return tuple(rows + columns + squares)
1
38
1 DEFAULT_TUPLES = _default_tuples()
2
```

```
167 def feature_indices(self, grid: np.ndarray, validate: bool = True) -> np.ndarray:
1     """Return the table index selected by each symmetry and tuple.
2
3     Pass validate=False in hot loops once the caller guarantees a legal board.
4     """
5     board = np.asarray(grid)
6     if board.shape != (BOARD_SIZE, BOARD_SIZE):
7         raise ValueError("grid must have shape (4, 4)")
8
9     ranks = self._ranks_checked(board) if validate else _RANK_LUT[board]
10
11     selected = ranks.reshape(-1)[self._symmetry_cells].astype(np.int64, copy=False)
12     return selected @ self._place_values
13
14 def value(self, grid: np.ndarray, validate: bool = True) -> float:
15     indices = self.feature_indices(grid, validate=validate)
16     table_numbers = np.arange(len(self.tuples), dtype=np.intp)[None, :]
17     return float(self.tables[table_numbers, indices].sum(dtype=np.float64))
18
```

```

12
13 def update(self, grid: np.ndarray, delta: float) -> None:
14     """Distribute delta evenly over every table lookup used by value()."""
15     try:
16         numeric_delta = float(delta)
17     except (TypeError, ValueError) as error:
18         raise TypeError("delta must be a finite real number") from error
19     if not np.isfinite(numeric_delta):
20         raise ValueError("delta must be finite")
21
22     indices = self.feature_indices(grid, validate=False)
23     amount_per_lookup = numeric_delta / indices.size
24     offsets = np.arange(len(self.tuples), dtype=np.int64) * TABLE_SIZE
25     np.add.at(self.tables.reshape(-1), (indices + offsets).reshape(-1), amount_per_lookup)
26

```

The excerpts show the 17 tuple patterns extracted from the board (4 rows, 4 columns, 9 overlapping 2×2 squares), the conversion of a board into lookup table indices across all 8 symmetries, and the paired `value()` and `update()` methods.

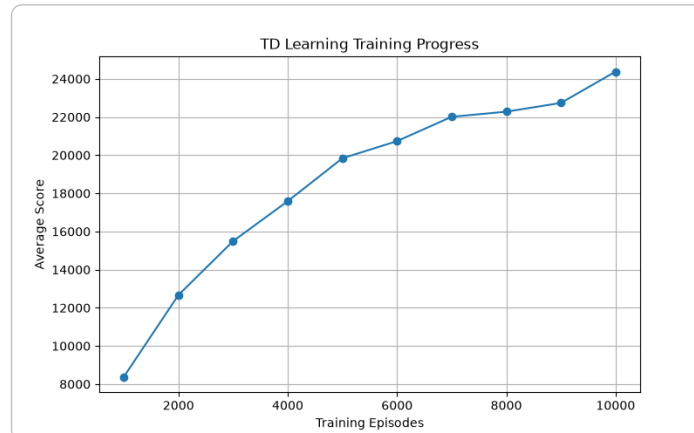
This replaces the hand-designed evaluation function from Milestone 1 with one that is learned. The symmetry handling is what makes it practical: the eight rotations and reflections of a board share the same tables, so one observed position writes into $8 \times 17 = 136$ table lookups at once and the agent generalizes from far fewer games. The `value()` and `update()` pair is the interface the TD trainer in Accomplishment 2 is built on.

Accomplishment 2: Implemented TD Learning Self-Play Training Pipeline

We implemented a Temporal Difference (TD) learning pipeline that improves the value function through self-play. The agent plays complete games on its own, picking at each turn the move with the highest estimated value, and after every move corrects its estimate of the position it just left using the reward earned and the value of the position that follows. Learned values are checkpointed every 1000 episodes, which lets training be resumed or evaluated later.

Each board cell is encoded as the exponent of its tile value (0 for empty, k for a tile of 2^k), which is what makes each four-cell tuple a base-16 lookup index. The reward for a move is the sum of the tiles created by merges on that move. The submitted value function was trained for 10,000 self-play episodes with a learning rate of $\alpha = 0.05$, applied to the afterstate reached after the player moves but before the random tile spawns.

Proof: training progress graph and the TD update in `training/td_learning.py`



```

def play_episode(self):
    """One self-play game with TD(0) updates over afterstates.

    V(afterstate) estimates FUTURE reward only, so the target for afterstate A1 is
    R2 + V(A2), where R2 is the reward of the move taken from the NEXT board.
    The current move's own reward is used for action selection, not for the target.
    """
    game = Game()
    action, afterstate, _reward, _value = self.evaluate_moves(game.board.grid)

    while action is not None:
        game.move(action)

        next_action, next_afterstate, _next_reward, target = self.evaluate_moves(
            game.board.grid
        )
        if next_action is None: # terminal: no future reward
            target = 0.0

        delta = target - self.network.value(afterstate, validate=False)
        self.network.update(afterstate, self.alpha * delta)

        action, afterstate = next_action, next_afterstate

    return game.score

```

The graph plots the average score over each block of 1000 training episodes, rising from roughly 8,400 to about 24,400 at 10,000. The code excerpt shows the TD(0) update over afterstates: the target is the reward of the following move plus the value of the next afterstate (0 at a terminal position), and the network is corrected by that error scaled by the learning rate. Since the value function starts at zero everywhere, all of this improvement comes from experience the agent generated itself, and the curve rising steadily rather than collapsing is the evidence that the updates are stable.

Accomplishment 3: Integrated RL Agent with Existing 2048 System and Completed Interactive UI

We integrated the reinforcement learning components with our existing 2048 environment (board movement, tile merging, random spawning, scoring, and termination), so the value function learns through self-play without any change to the core game mechanics, and we completed the Pygame interface: a main menu for selecting the agent, the 4×4 board, score and best score, and tile animations for movement and merging.

The search follows the structure of the game rather than a turn-based game tree. The player is a max node over the legal moves and the environment is a chance node over every empty cell, with a 2 spawning with probability 0.9 and a 4 with probability 0.1. The two alternate, and there is no min node, because nothing in 2048 plays against the agent.

Proof: the evaluation function that lets one search use either value function, and a table comparing the Random baseline, the heuristic Expectimax agent, and the Expectimax agent using the learned value function

```

8
7      # Learned value function or the previous heuristic can be used.
6      def evaluate(self, state):
5          if self.network is not None:
4              return self.network.value(state, validate=False)
3          empty = len(empty_cells(state))
2          max_tile = np.max(state)
1          return (100 * empty) + max_tile
98

```

Agent	Games	Avg Score	Median	Best	Best Tile	Reached 2048
Random	15	992.5	720	2,772	256	0/15
Expectimax (heuristic)	15	9,974	7,184	26,768	2048	1/15
Expectimax + RL	15	48,487	37,880	78,224	4096	15/15

All three agents played 15 games on the same environment, with the RL agent using the trained value function inside the same Expectimax search at base depth 2, deepening to 3 when at most two cells are empty.

The learned value function outperforms the hand-written heuristic by roughly $4.9\times$ and reaches the 2048 tile in every game, where the heuristic reaches it once in 15 and the random baseline never does. Search and learning each contribute: the same trained network selecting moves greedily, with no lookahead, averages 21,697 and reaches 2048 six times in 15 games, against 48,487 and 15/15 inside the search. Neither component alone accounts for the result, which is why the heuristic Expectimax is kept as a baseline rather than replaced.

Proof for UI: [UI Demonstration Video](#)

The video shows the trained agent playing through the finished interface end to end: the menu selects an agent, the board updates move by move under that agent's decisions, and the score rises as merges happen. It demonstrates progress because the learned value function, the search, and the interface are running together as one system rather than as separate pieces.

Challenges

The main challenge in Milestone 2 was choosing how to represent and train the value function. The 2048 environment makes this difficult: tile spawns are stochastic, the number of reachable board configurations is enormous, and any representation has to be cheap enough to evaluate inside a search. We considered a neural network first and settled on an N-tuple network with TD learning, which is far cheaper per evaluation and needs no architecture tuning.

Two problems inside the pipeline took several attempts to get right. The first was where the reward belongs in the update. An early version added the reward of the current move to its own target, which double counts it: the value of an afterstate estimates future reward only, so the target has to use the reward of the move taken from the next board. Fixing this changed the quantity being learned, not just the numbers. The second was the learning rate. The update is spread across all 136 lookups selected by the symmetric tuples, so the effective step per

weight is much smaller than the nominal rate, and our first choice of 0.01 was learning far more slowly than it appeared to. We settled it by measurement: over 10,000 episodes, 0.01 averaged about 16,000 and 0.05 about 24,500, so the submitted agent uses 0.05.

Changes from Original Plan

Our scope and objective are unchanged, but two implementation decisions were changed from the proposal. We planned to implement a neural-network Q-learning agent in PyTorch, but after further research into reinforcement learning for 2048 we replaced the neural network with an N-tuple network, and action-value Q-learning with afterstate TD(0) state-value learning. The N-tuple network is far cheaper to evaluate and requires no architecture design, and afterstate values fit this game better because the player's move is deterministic and only the spawn that follows is random: instead of a value for every board and action pair, the agent learns one value per resulting position, reached after the move but before the spawn.

Two smaller departures follow from the same decision. The feature-based Q-learning agent listed in the proposal as the minimal viable system was not built, because the N-tuple network learns board features directly from experience. Expectimax also changed role: the proposal treated it purely as a baseline, and it is now also the policy acting on the learned value function, with the heuristic version kept as the baseline.

GitHub repo link: <https://github.com/praneetkb/2048-Master>